การเขียนโปรแกรมสำหรับดุลสมการเคมี

ขั้นตอนวิธีในการดุลสมการเคมีโดยใช้ความรู้เรื่องการแก้ระบบสมการเชิงเส้น

1. สมมุติให้เลขสัมประสิทธิ์หน้าสารแต่ละชนิดมีค่าเป็น $x_1, x_2, x_3, \cdots, x_n$ ตามลำดับ เมื่อ n คือจำนวนของสารที่ปรากฏในสมการนั้น

2. พิจารณาว่ามีธาตุใดบ้างที่ปรากฏในสมการ จากนั้นสร้างสมการของแต่ละธาตุตามเงื่อนไขที่กล่าวไว้ว่า "จำนวนอะตอมของแต่ละธาตุในสารตั้งต้นและผลิตภัณฑ์หลังดุลสมการเคมีจะมีค่าเท่ากัน"

3. เนื่องจากเลขสัมประสิทธิ์หน้าสารแต่ละชนิดเป็นอัตราส่วน $x_1 : x_2 : x_3 : \cdots : x_n$ ทำให้ระบบสมการที่ได้จากข้อ 2 มีคำตอบหลายชุด ดังนั้นจึงต้องเลือกแทนตัวแปรใดตัวแปรหนึ่งให้เป็น 1 เพื่อ จำกัดคำตอบของระบบสมการนี้ให้เหลือเพียง 1 ชุด

4. ดำเนินการแก้ระบบสมการเชิงเส้นหลายตัวแปร จะได้คำตอบ $(x_1, x_2, x_3, \cdots, x_n)$ ที่สอดคล้องกับระบบสมการจำนวน 1 ชุด

5. หากเลขสัมประสิทธิ์ของสารแต่ละชนิดยังไม่เป็นจำนวนเต็ม จะต้องดำเนินการคูณสัมประสิทธิ์ทุกตัวด้วย ค.ร.น. ของ $x_1, x_2, x_3, \cdots, x_n$ ที่ได้เพื่อทำให้เลขสัมประสิทธิ์เป็นอัตราส่วนอย่างต่ำและเป็นจำนวนเต็ม (สุวิจักขณ์ ดาวเรือง, 2566)

<u>ตัวอย่างการนำไปใช้ :</u> ดุลสมการเคมีต่อไปนี้

$$S + H_2SO_4 \longrightarrow SO_2 + H_2O$$

<u>วิธีทำ :</u> สมมุติให้เลขสัมประสิทธิ์ ของแต่ละสารเป็น x_1, x_2, x_3 และ x_4 ตามลำดับ

$$x_1S + x_2H_2SO_4 \longrightarrow x_3SO_2 + x_4H_2O$$

โดยใช้หลักการที่ว่า "จำนวนอะตอมของแต่ละธาตุในสารตั้งต้นและผลิตภัณฑ์หลังดุลสมการเคมีจะมีค่าเท่ากัน" จะเห็นว่าสมการนี้ประกอบไปด้วยธาตุ S, H และ O พิจารณาแต่ละธาตุดังนี้

$$S: \quad x_1 + x_2 = x_3 \qquad \text{———}(1)$$

$$H: \quad 2x_2 = 2x_4 \qquad \text{———}(2)$$

$$O: \quad 4x_2 = 2x_3 + x_4 \qquad \text{———}(3)$$

จะพบว่ามี 3 สมการ 4 ตัวแปร ซึ่งทำให้ระบบสมการนี้มีคำตอบได้หลายชุด ดังนั้นจึงเลือกแทนตัวแปรใดตัวแปรหนึ่งให้เป็น 1 เพื่อจำกัดคำตอบของระบบสมการนี้ให้เหลือเพียง 1 ชุด ในที่นี้เลือกแทน $x_2 = 1$

ดังนั้นจาก (2) จะได้

$$2(1) = 2x_4$$

$$x_4 = 1$$

แทน x_2 และ x_4 ใน (3) จะได้

$$4(1) = 2x_3 + 1$$
$$x_3 = \frac{3}{2}$$

แทน x_2 และ x_3 ใน (1) จะได้

$$x_1 + 1 = \frac{3}{2}$$
$$x_1 = \frac{1}{2}$$

ดังนั้นคำตอบชุดหนึ่งของระบบสมการคือ $(x_1,\ x_2,\ x_3,\ x_4) = \left(\frac{1}{2},\ 1,\ \frac{3}{2},\ 1\right)$ แทนในสมการเคมี จะได้

$$\frac{1}{2}S + H_2SO_4 \longrightarrow \frac{3}{2}SO_2 + H_2O$$

จากนั้นคูณ 2 ซึ่งเป็น ค.ร.น. ของ $\left(\frac{1}{2},\ 1,\ \frac{3}{2},\ 1\right)$ ทั้งสมการเพื่อกำจัดตัวส่วน เพราะฉะนั้นสมการที่ดุลแล้วคือ

$$S + 2H_2SO_4 \longrightarrow 3SO_2 + 2H_2O$$ ■

การหาคำตอบของระบบสมการเชิงเส้นโดยใช้เมทริกซ์แต่งเติม

ในการหาคำตอบของระบบสมการเชิงเส้นนั้น หนึ่งในวิธีที่สามารถใช้ได้คือ การใช้ **เมทริกซ์แต่งเติม (augmented matrix)** ซึ่งมีบทนิยามดังนี้

สำหรับระบบสมการเชิงเส้นที่ประกอบด้วย m สมการ n ตัวแปร ต่อไปนี้

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m \end{aligned}$$

จะเรียกเมทริกซ์ที่อยู่ในรูป

$$\left[A \mid B\right] = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{array}\right]$$

ว่า เมทริกซ์แต่งเติมของระบบสมการ

เนื่องจากกระบวนการต่อไปนี้ไม่ทำให้คำตอบของระบบสมการเชิงเส้นเปลี่ยนแปลง

1. สลับสองสมการ

2. คูณสมการใดสมการหนึ่งด้วยค่าคงตัวที่ไม่ใช่ศูนย์

3. คูณสมการหนึ่งด้วยค่าคงตัวที่ไม่ใช่ศูนย์แล้วนำไปบวกกับอีกสมการหนึ่ง

ดังนั้นเพื่อความสะดวกในการแก้ระบบสมการเชิงเส้นหลายตัวแปรจึงนิยมเขียนระบบสมการในรูปเมทริกซ์แต่งเติม แล้วใช้การดำเนินการตามแถว ซึ่งมี 3 แบบต่อไปนี้กับเมทริกซ์

แบบที่ 1 สลับแถวที่ i และแถวที่ j ของเมทริกซ์ ซึ่งจะแทนด้วยสัญลักษณ์ $R_i \leftrightarrow R_j$

แบบที่ 2 คูณสมาชิกในแถวที่ i ด้วยค่าคงตัว c เมื่อ $c \neq 0$ ซึ่งจะแทนด้วยสัญลักษณ์ cR_i

แบบที่ 3 คูณสมาชิกในแถวที่ i ด้วยค่าคงตัว c เมื่อ $c \neq 0$ แล้วนำไปบวกกับสมาชิกในแถวที่ j เมื่อ $i \neq j$ ซึ่งจะแทนด้วยสัญลักษณ์ $cR_i + R_j$ (แทนผลลัพธ์นี้ในแถวที่ j)

เรียกดำเนินกับเมทริกซ์แต่ละแบบนี้ว่า **การดำเนินการตามแถวขั้นมูลฐาน** (elementaty row operation) โดยเราจะดำเนินการกับเมทริกซ์แต่งเติม จนได้เมทริกซ์แต่งเติมที่มีลักษณะดังนี้

$$\left[\begin{array}{cccc|c} 1 & 0 & \cdots & 0 & k_1 \\ 0 & 1 & \cdots & 0 & k_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & k_m \end{array}\right]$$

เมื่อแปลงเป็นระบบสมการจะได้

$$\begin{array}{rcl} x_1 + 0x_2 + \cdots + 0x_n &=& k_1 \\ 0x_1 + x_2 + \cdots + 0x_n &=& k_2 \\ &\vdots& \\ 0x_1 + 0x_2 + \cdots + x_n &=& k_m \end{array} \longrightarrow \begin{array}{rcl} x_1 &=& k_1 \\ x_2 &=& k_2 \\ &\vdots& \\ x_n &=& k_m \end{array}$$

ทำให้ได้คำตอบของระบบสมการนี้เป็น $(x_1, x_2, x_3, \cdots, x_n) = (k_1, k_2, k_3, \cdots, k_n)$ ตามต้องการ

การแก้ระบบสมการด้วยวิธีนี้อาจเรียกอีกแบบว่า **การกำจัดแบบเกาส์ (Gaussian elimination)**

<u>ตัวอย่างการนำไปใช้ :</u> จงหาคำตอบของระบบสมการเชิงเส้นต่อไปนี้ (โจทย์เดียวกันกับหน้าแรก)

$$\begin{aligned} x_1 + x_2 &= x_3 \\ 2x_2 &= 2 \\ 4x_2 &= 2x_3 + 1 \end{aligned}$$

<u>วิธีทำ :</u> จากโจทย์สามารถจัดรูประบบสมการได้ดังนี้

$$\begin{aligned} x_1 + x_2 - x_3 &= 0 \\ 0x_1 + 2x_2 + 0x_3 &= 2 \\ 0x_1 + 4x_2 - 2x_3 &= 1 \end{aligned}$$

ซึ่งสามารถนำไปเขียนเป็นเมทริกซ์แต่งเติมได้ดังนี้

$$\left[\begin{array}{ccc|c} 1 & 1 & -1 & 0 \\ 0 & 2 & 0 & 2 \\ 0 & 4 & -2 & 1 \end{array}\right]$$

ใช้การดำเนินการตามแถวเพื่อแปลงเมทริกซ์แต่งเติมได้ดังนี้

$$\left[\begin{array}{ccc|c} 1 & 1 & -1 & 0 \\ 0 & 2 & 0 & 2 \\ 0 & 4 & -2 & 1 \end{array}\right] \sim \left[\begin{array}{ccc|c} 1 & 1 & -1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 4 & -2 & 1 \end{array}\right] \begin{array}{l} \\ \frac{1}{2}R_2 \\ \\ \end{array}$$

$$\sim \left[\begin{array}{ccc|c} 1 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & -2 & -3 \end{array}\right] \begin{array}{l} -R_2 + R_1 \\ \\ -4R_2 + R_3 \end{array}$$

$$\sim \left[\begin{array}{ccc|c} 1 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & \frac{3}{2} \end{array}\right] \begin{array}{l} \\ \\ -\frac{1}{2}R_3 \end{array}$$

$$\sim \left[\begin{array}{ccc|c} 1 & 0 & 0 & \frac{1}{2} \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & \frac{3}{2} \end{array}\right] \begin{array}{l} R_3 + R_1 \\ \\ \\ \end{array}$$

เมื่อแปลงเมทริกซ์แต่งเติมนี้ให้อยู่ในรูประบบสมการจะได้ $x_1 = \frac{1}{2}$, $x_2 = 1$ และ $x_3 = \frac{3}{2}$

ดังนั้น $\left(\frac{1}{2}, 1, \frac{3}{2}\right)$ เป็นคำตอบของระบบสมการ ■

การเขียนโปรแกรมภาษา Javascript เพื่อดุลสมการ

1. เก็บอะตอมที่ปรากฏในสารตั้งต้นและผลิตภัณฑ์ไว้ใน Array ที่มีชื่อว่า **atomList**
2. เก็บสารที่ปรากฏในฝั่งสารตั้งต้นไว้ใน Array ที่มีชื่อว่า **reactantList**
3. เก็บสารที่ปรากฏในฝั่งผลิตภัณฑ์ไว้ใน Array ที่มีชื่อว่า **productList**
4. สร้าง Array เพื่อเก็บสัมประสิทธิ์ของระบบสมการ ในขณะเริ่มต้นให้มีสมาชิกเป็น Array ว่าง (จำนวนของ Array ว่างที่อยู่ใน Array สัมประสิทธิ์ เท่ากับจำนวนชนิดของอะตอมที่ปรากฏในสมการเคมี) โดย Array นี้มีชื่อว่า **coffMatrix**
5. สร้าง Array เพื่อเก็บพจน์ค่าคงตัวของระบบสมการ ในขณะเริ่มต้นให้เป็น Array ว่าง โดย Array นี้มีชื่อว่า **constantMatrix**

ยกตัวอย่างเช่นจากสมการเคมี $C_6H_{12}O_6 + O_2 \longrightarrow CO_2 + H_2O$

จะได้ **atomList** = [C, H, O]

reactantList = [C6H12O6, O2]

productList = [CO2, H2O]

coffMatrix = [[ ], [ ], [ ]]

constantMatrix = [ ]

6. ดำเนินการตรวจสอบอะตอมตัวที่ i ที่อยู่ใน **atomList** เมื่อ $i \in \{0,1,2,\cdots,n-1\}$ เมื่อ n เป็นจำนวนสมาชิกของ **atomList** ว่าอยู่ใน **reactantList** และ **productList** จำนวนกี่อะตอม

- ถ้าอะตอมนั้นอยู่ใน **reactantList** ให้ใส่แล้วใส่<u>จำนวนอะตอม</u>ลงใน Array ว่างตัวที่ i ซึ่งอยู่ใน **coffMatrix** แต่ถ้าไม่มีให้ใส่ค่า 0 ลงใน Array ว่างตัวที่ i ซึ่งอยู่ใน **coffMatrix**
- จากนั้นตรวจสอบว่าถ้าอะตอมนั้นอยู่ใน **productList** ให้ใส่แล้วใส่<u>ค่าติดลบของจำนวนอะตอม</u>ลงใน Array ว่างตัวที่ i ซึ่งอยู่ใน **coffMatrix** แต่ถ้าไม่มีให้ใส่ค่า 0 ลงใน Array ว่างตัวที่ i ซึ่งอยู่ใน **coffMatrix**
- ยกเว้นอะตอมที่นำไปตรวจสอบกับสารตัวสุดท้ายใน **productList** ให้ใส่<u>จำนวนอะตอม</u>ลงใน **constantMatrix**

ยกตัวอย่างเช่นจากสมการเคมี $C_6H_{12}O_6 + O_2 \longrightarrow CO_2 + H_2O$

- เมื่อตรวจสอบอะตอม C จะได้ **coffMatrix** = [[6, 0, -1], [ ], [ ]]
  constantMatrix = [0]
- จากนั้นตรวจสอบอะตอม H จะได้ **coffMatrix** = [[6, 0, -1], [12, 0, 0], [ ]]
  constantMatrix = [0, -2]

- จากนั้นตรวจสอบอะตอม O จะได้ **coffMatrix** = [[6, 0, -1], [12, 0, 0], [6, 2, -2]]

constantMatrix = [0, 2, 1]

จากตัวอย่างข้างต้น จะเห็นว่า **constantMatrix** และ **coffMatrix** ซึ่งเป็น Array ใน Javascript นั้น สามารถเทียบได้กับเมทริกซ์ที่มีมิติ 3×3 และ 3×1 ตามลำดับดังนี้

$$\text{ConstantMatrix} = \begin{bmatrix} 6 & 0 & -1 \\ 12 & 0 & 0 \\ 6 & 2 & -2 \end{bmatrix} \text{ และ } \text{CoffMatrix} = \begin{bmatrix} 0 \\ 2 \\ 1 \end{bmatrix}$$

โดยสามารถเขียนเมทริกซ์แต่งเติมจากเมทริกซ์ทั้งสองนี้ได้ดังนี้ $\left[\begin{array}{ccc|c} 6 & 0 & -1 & 0 \\ 12 & 0 & 0 & 2 \\ 6 & 2 & -2 & 1 \end{array}\right]$

7. เขียนโปรแกรมโดยใช้หลักการการดำเนินการตามแถวขั้นมูลฐานดำเนินการกับเมทริกซ์แต่งเติม จนได้ Array ที่เป็นคำตอบของระบบสมการออกมา เช่น จากตัวอย่างในข้อ 5 เมื่อดำเนินการแปลงเมทริกซ์แต่งเติมเสร็จสิ้น จะได้

เมทริกซ์แต่งเติม $\left[\begin{array}{ccc|c} 1 & 0 & 0 & \frac{1}{6} \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{array}\right]$ ซึ่งทำให้ได้ Array ที่เป็นคำตอบของสมการคือ $\left[\frac{1}{6}, 1, 1\right]$

8. push เลข 1 เข้าไปใน Array ที่เป็นคำตอบของระบบสมการ (เหตุผลเพราะจากขั้นตอนวิธีในการดุลสมการเคมีโดยใช้ความรู้เรื่องการแก้ระบบสมการเชิงเส้นในหน้าที่ 1 ที่ต้องมีการเลือกแทนตัวแปรใดตัวแปรหนึ่งให้เป็น 1 เพื่อจำกัดคำตอบของระบบสมการนี้ให้เหลือเพียง 1 ชุด ซึ่งในที่นี้เพื่อความสะดวกในการเขียนโปรแกรมจึงเลือกให้สัมประสิทธิ์ของสารผลิตภัณฑ์ตัวสุดท้ายมีค่าเป็น 1 เสมอ)

จากตัวอย่างในข้อ 7 จะได้ Array ที่เป็นเก็บเลขสัมประสิทธิ์ของสารแต่ละตัวเรียงจากซ้ายไปขวาคือ $\left[\frac{1}{6}, 1, 1, 1\right]$

9. ทำให้เลขสัมประสิทธิ์ใน Array เป็นอัตราส่วนอย่างต่ำและเป็นจำนวนเต็ม

จากตัวอย่างในข้อ 8 จะได้ Array ที่เป็นเก็บเลขสัมประสิทธิ์ของสารแต่ละตัวซึ่งเป็นอัตราส่วนอย่างต่ำและเป็นจำนวนเต็ม เรียงจากซ้ายไปขวาคือ $[1, 6, 6, 6]$

10. เขียนโปรแกรมสำหรับแสดงผลสมการเคมีที่ดุลแล้ว โดยใช้สัมประสิทธิ์ใน Array ที่ได้ในข้อ 9

จากตัวอย่างในข้อ 9 จะได้สมการที่ดุลแล้วที่นำไปแสดงผลคือ $C_6H_{12}O_6 + 6O_2 \longrightarrow 6CO_2 + 6H_2O$

โปรแกรมภาษา Javascript ที่ใช้ในการดุลสมการเคมี

```
let reactantDiv = document.querySelector(".reactant");
let productDiv = document.querySelector(".product");
let addReactantButton = document.querySelector(".left-button");
let addProductButton = document.querySelector(".right-button");
let solveButton = document.querySelector(".solve-button");
let resetButton = document.querySelector(".reset-button");
let answerBox = document.querySelector(".answer");

function addReactant() {
    reactantDiv.innerHTML += ' + <input class="left-input" type="text"
/>';
}

function addProduct() {
    productDiv.innerHTML += ' + <input class="right-input" type="text"
/>';
}

function reset() {
    reactantDiv.innerHTML = '<input class="left-input" type="text" />';
    productDiv.innerHTML = '<input class="right-input" type="text" />';
    answerBox.innerHTML = '<p class="equation">สมการที่ดุลแล้วจะถูกแสดงที่นี่</p>';
    MathJax.Hub.Queue(["Typeset", MathJax.Hub, "answerBox"]);
}

function separateTextFromNumbers(text) {
    // ตัดตัวเลข
    const textWithoutNumbers = text.replace(/\d/g, '');
    // แยกตัวอักษรพิมพ์เล็ก/ใหญ่
    const separatedText = textWithoutNumbers.match(/[A-Z][a-z]*/g);
    return separatedText;
}

function checkAtom(atom, molecule) {
    while (molecule.includes(atom)) {
        const index = molecule.indexOf(atom);
        const nextChar = molecule[index + atom.length];

        if (atom.length === 1 && nextChar && !/\d/.test(nextChar)) {
            if (nextChar.toLowerCase() === nextChar) {
                molecule = molecule.slice(0, index) + molecule.slice(index + 2);
            } else {
```

```
                return 1;
            }
        } else {
            let numStr = '';
            let i = index + atom.length;

            while (i < molecule.length && /\d/.test(molecule[i])) {
                numStr += molecule[i];
                i++;
            }

            return parseInt(numStr) || 1;
        }
    }
    return 0;
}

function diagonalize(M) {
    var m = M.length;
    var n = M[0].length;
    for (var k = 0; k < Math.min(m, n); ++k) {
        // Find the k-th pivot
        i_max = findPivot(M, k);
        if (M[i_max, k] == 0)
            throw "matrix is singular";
        swap_rows(M, k, i_max);
        // Do for all rows below pivot
        for (var i = k + 1; i < m; ++i) {
            // Do for all remaining elements in current row:
            var c = M[i][k] / M[k][k];
            for (var j = k + 1; j < n; ++j) {
                M[i][j] = M[i][j] - M[k][j] * c;
            }
            // Fill lower triangular matrix with zeros
            M[i][k] = 0;
        }
    }
}

function findPivot(M, k) {
    var i_max = k;
    for (var i = k + 1; i < M.length; ++i) {
        if (Math.abs(M[i][k]) > Math.abs(M[i_max][k])) {
            i_max = i;
        }
```

```
    }
    return i_max;
}

function swap_rows(M, i_max, k) {
    if (i_max != k) {
        var temp = M[i_max];
        M[i_max] = M[k];
        M[k] = temp;
    }
}

function makeM(A, b) {
    for (var i = 0; i < A.length; ++i) {
        A[i].push(b[i]);
    }
}

function substitute(M) {
    var m = M.length;
    for (var i = m - 1; i >= 0; --i) {
        var x = M[i][m] / M[i][i];
        for (var j = i - 1; j >= 0; --j) {
            M[j][m] -= x * M[j][i];
            M[j][i] = 0;
        }
        M[i][m] = x;
        M[i][i] = 1;
    }
}

function extractX(M) {
    var x = [];
    var m = M.length;
    var n = M[0].length;
    for (var i = 0; i < m; ++i) {
        x.push(M[i][n - 1]);
    }
    return x;
}

function solve(A, b) {
    makeM(A, b);
    diagonalize(A);
    substitute(A);
```

```
    var x = extractX(A);
    return x;
}

function shuffle(array) {
    let currentIndex = array.length, randomIndex;

    // While there remain elements to shuffle.
    while (currentIndex > 0) {

        // Pick a remaining element.
        randomIndex = Math.floor(Math.random() * currentIndex);
        currentIndex--;

        // And swap it with the current element.
        [array[currentIndex], array[randomIndex]] = [
            array[randomIndex], array[currentIndex]];
    }

    return array;
}

function decimalToFraction(decimal) {
    const tolerance = 1.0e-10;
    const sign = Math.sign(decimal); // เก็บเครื่องหมายของจำนวน
    decimal = Math.abs(decimal); // ทำให้จำนวนเป็นบวก

    let h1 = 1;
    let h2 = 0;
    let k1 = 0;
    let k2 = 1;
    let b = decimal;
    do {
        const a = Math.floor(b);
        const aux = h1;
        h1 = a * h1 + h2;
        h2 = aux;
        const aux2 = k1;
        k1 = a * k1 + k2;
        k2 = aux2;
        b = 1 / (b - a);
    } while (Math.abs(decimal - h1 / k1) > decimal * tolerance);

    const numerator = sign * h1; // นำเครื่องหมายมาใช้
    const denominator = k1;
```

```
    return numerator + "/" + denominator;
}

function convertToSubscript(inputList) {
    let subscriptList = [];

    for (let item of inputList) {
        let subscriptItem = "";
        let currentDigit = "";

        for (let char of item) {
            if (!isNaN(char)) {
                currentDigit += char;
            } else {
                if (currentDigit) {
                    subscriptItem += `_{${currentDigit}}`;
                    currentDigit = "";
                }
                subscriptItem += char;
            }
        }

        if (currentDigit) {
            subscriptItem += `_{${currentDigit}}`;
        }

        subscriptList.push(subscriptItem);
    }

    return subscriptList;
}

function balancing() {
    let reactants = document.querySelectorAll(".left-input");
    let products = document.querySelectorAll(".right-input");
    let reactantList = [];
    let productList = [];

    reactants.forEach((box) => {
        reactantList.push(box.value);
    });
    products.forEach((box) => {
        productList.push(box.value);
    });
```

```
    const inputList = [...reactantList, ...productList];

    while (true) {
        let atomList = [];
        let coffMatrix = [];
        let constantMatrix = [];

        for (const item of inputList) {
            atomList = [...atomList, ...separateTextFromNumbers(item)];
        }

        atomList = [...new Set(atomList)]
        shuffle(atomList);
        popList = [];

        if (inputList.length - atomList.length > 1) {
            answerBox.innerHTML = "<p class='equation'>เลขสัมประสิทธิ์ที่สอดคล้องกับสมการนี้มีหลายได้หลาย
ชุด</p>";
            return;
        }

        while (inputList.length - atomList.length !== 1) {
            popList.push(atomList.pop());
        }

        for (let i = 0; i < atomList.length; i++) {
            coffMatrix.push([]);
        }

        for (const atom of atomList) {
            for (const r of reactantList) {
                coffMatrix[atomList.indexOf(atom)].push(checkAtom(atom, r));
            }

            for (const p of productList) {
                if (productList.indexOf(p) === productList.length - 1) {
                    constantMatrix.push(checkAtom(atom, p));
                } else {
                    coffMatrix[atomList.indexOf(atom)].push(-checkAtom(atom, p));
                }
            }
        }

        var x = solve(coffMatrix, constantMatrix);
```

```
        x.push(1);

        let xFrac = [];
        let Answer = true;

        for (const ans of x) {
            if (isNaN(ans)) {
                Answer = false;
                break;
            } else {
                xFrac.push(decimalToFraction(ans));
            }
        }

        // Extract denominators and calculate LCM
        const Denominators = xFrac.map(frac => parseInt(frac.split('/')[1]));
        // Function to calculate greatest common divisor
        function gcd(a, b) {
            return b === 0 ? a : gcd(b, a % b);
        }
        const lcm = Denominators.reduce((a, b) => a * b / gcd(a, b), 1);

        // Convert string fractions to decimals
        const decimals = xFrac.map(frac => eval(frac));

        // Multiply decimals by LCM to get coffAnswer
        const coffAnswer = decimals.map(decimal => Math.round(decimal * lcm));

        let subReactantList = convertToSubscript(reactantList);
        let subProductList = convertToSubscript(productList);

        // Create the chemical equation
        let equation = coffAnswer.slice(0, subReactantList.length).map((coff,
index) => {
            let reactant = subReactantList[index];
            return `${coff !== 1 ? coff : ''}${reactant}`;
        }).join(" + ");

        // Add the equals sign
        equation += " \\longrightarrow ";

        // Add the productList part
        equation += coffAnswer.slice(subReactantList.length).map((coff, index) =>
{
            let product = subProductList[index];
```

```
            return `${coff !== 1 ? coff : ''}${product}`;
        }).join(" + ");

        latexEquation = "<p class='equation'>$$ " + equation + " $$</p>";

        if (Answer) {
            let leftCoff = x.slice(0, reactantList.length);
            let rightCoff = x.slice(reactantList.length);
            let popCheck = 0;

            for (const e of popList) {
                for (const r of reactantList) {
                    popCheck += checkAtom(e, r) *
leftCoff[reactantList.indexOf(r)];
                }

                for (const p of productList) {
                    popCheck -= checkAtom(e, p) *
rightCoff[productList.indexOf(p)];
                }

                if (Math.floor(Math.abs(popCheck)) !== 0) {
                    answerBox.innerHTML = "<p class='equation'>ไม่มีเลขสัมประสิทธิ์ที่สอดคล้องกับ
สมการนี้ (สมการไม่ถูกต้อง)</p>";
                    return;
                }
            }

            answerBox.innerHTML = latexEquation;
            MathJax.Hub.Queue(["Typeset", MathJax.Hub, "answerBox"]);
            break;
        }
    }
}

addReactantButton.addEventListener('click', addReactant);
addProductButton.addEventListener('click', addProduct);
resetButton.addEventListener('click', reset);
solveButton.addEventListener('click', balancing);
```